

Criando Container com Docker

Prof: Vivianne Melo

Docker

O Docker é uma plataforma que permite criar, executar e gerenciar aplicações em ambientes isolados chamados containers.

Um container reúne a aplicação e os recursos necessários para que ela possa ser executada de forma organizada e padronizada.

No projeto de vocês, a utilização de containers facilita bastante a execução da aplicação em diferentes computadores.

Sem Docker, se fosse necessário executar o projeto em outro computador, seria preciso preparar todo o ambiente novamente, por exemplo:

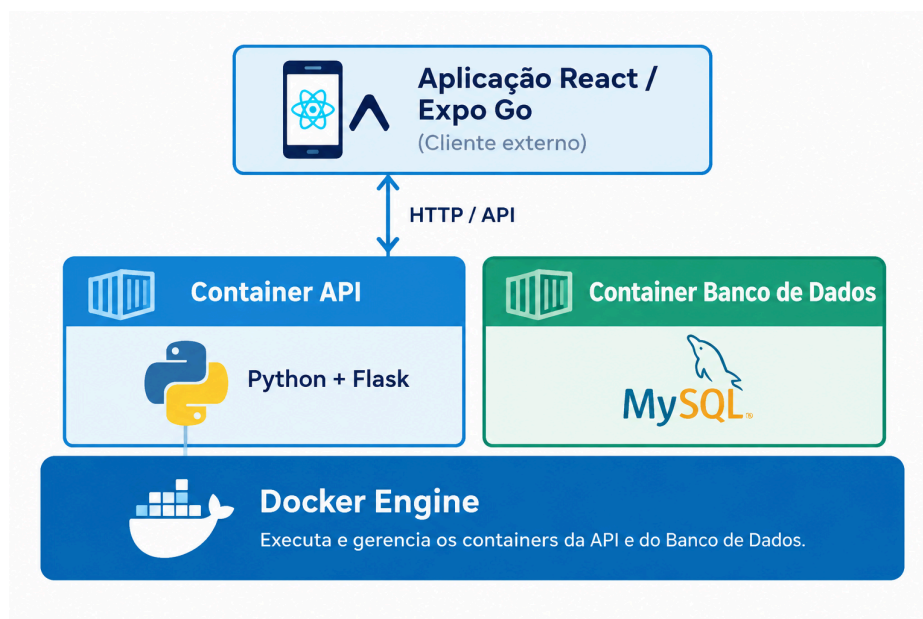
- Instalar o **Python e o Flask**;
- Instalar as **bibliotecas e dependências** do projeto;
- Instalar e configurar o **MySQL**;
- Criar e configurar o **banco de dados**;
- Ajustar portas e outras configurações necessárias.

Com Docker, podemos descrever o ambiente necessário em arquivos de configuração.

Esses arquivos informam:

- Qual tecnologia será utilizada;
- Qual versão será utilizada;
- Quais dependências são necessárias;
- Como os serviços devem funcionar;
- Como os containers devem se comunicar.

ARQUITETURA DO NOSSO PROJETO



Cada container será responsável por uma função, o da API por: Python, Flask, Bibliotecas, Código da API. Já o do banco por: MySQL, Banco de dados, Tabelas Dados.

DOCKER HUB

É um repositório online onde várias imagens docker são disponibilizadas, elas podem ser utilizadas para a criação de container

Exemplo: Imagem do MySQL
mysql:8.0

Exemplo: Imagem do Python
python:3.12-slim

IMAGEM DOCKER

É um modelo utilizado para criar um container. Ela pode conter tudo o que é necessário para preparar um determinado ambiente, como:

- Sistema base;
- Python;
- Bibliotecas;
- Dependências;

- Arquivos da aplicação;
- Configurações.

Existem imagens prontas, como: python:3.12-slim, mysql:8.0 e também conseguimos criar as nossas imagens do zero.

Observação: Antes de criar nossa imagem, precisamos verificar qual versão do Python o projeto utiliza.

Digitando no terminal do IDE: **python --version** ou **py --version**

DOCKERFILE

É um arquivo de texto que contém as instruções necessárias para construir uma imagem Docker. Nele definimos a imagem base, os arquivos, as dependências e como a aplicação será executada.

Exemplo

Dentro do seu projeto vamos criar um arquivo **Dockerfile**.

1º Passo: Vamos definir a imagem base que utilizaremos.

Exemplo: FROM python:3.12-slim

2º Passo: Vamos definir a pasta da aplicação.

Exemplo: WORKDIR /app

3º Passo: Copiando o requirements.txt

Exemplo: COPY requirements.txt .

4º Passo: Instalando as dependências.

Exemplo: RUN pip install -r requirements.txt

5º Passo: Copia o arquivo app.py do projeto para a pasta /app dentro da imagem.

Exemplo: COPY app.py .

6º Passo: Definindo como a aplicação será iniciada

Exemplo: CMD ["python", "app.py"]

Exemplo Completo

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY app.py .

CMD ["python", "app.py"]
```

Observação: A **imagem** é o modelo pronto que contém tudo o que a aplicação precisa. O **Container** é a execução dessa imagem.

Exemplo de criação da Imagem

Acesse o seu projeto do tcc e execute:

1º Passo: docker build -t almox-api .

O **docker build** utilizamos para construir uma imagem, já **-t almox-api** estamos definindo o nome da imagem e o **.** usamos para dizer ao docker para usar a pasta atual como contexto da construção.

Docker Compose

O projeto de vocês não possui apenas a API. Ele precisa de:

- uma API Flask para executar o sistema;
- um MySQL para armazenar os dados.

O docker compose é uma ferramenta que permite definir vários serviços da aplicação em um único arquivo (o docker-compose.yml) e gerenciar esses containers juntos (subir, pausar, reiniciar e derrubar todos com um único comando).

Exemplo

No docker-compose.yml, vamos criar dois serviços:

api: Responsável por executar nossa aplicação Flask.

db: responsável por executar o MySQL.

Os dois serviços ficarão na mesma rede criada pelo Docker Compose, permitindo que eles se comuniquem.

Observação: api e db são os nomes dos serviços dentro do Compose. No flask teremos **host="db"**

Não usaremos **localhost** porque dentro do ambiente Docker, localhost representa o próprio container.

Exemplo Criando o Docker Compose

```
services:
  api:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - db
  db:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: almox
      MYSQL_ROOT_PASSWORD: "mysql_root"
    ports:
      - "3306:3306"
    volumes:
      - ./banco.sql:/docker-entrypoint-initdb.d/banco.sql
```

- **api:** Serviço responsável pela aplicação Flask.
- **build:** . : O Docker Compose procura o Dockerfile na pasta atual e constrói a imagem da API.
- **ports:** "5000:5000": A porta 5000 do computador é ligada à porta 5000 dentro do container.
- **depends_on:** db: Indica que o serviço api depende do serviço db.
- **db:** Serviço responsável pelo MySQL.
- **image:** mysql:8.0: Utiliza a imagem do MySQL 8.0.
- **environment:** define o banco almox e a senha do usuário root.
- **volumes:** monta o banco.sql no diretório de inicialização do MySQL para que ele seja executado quando o banco for inicializado.

Mudança Muita Atenção

No seu app.py deve ter algo assim na conexão do banco de dados:

```
def conectar_banco():
    return mysql.connector.connect(
        host="127.0.0.1",      # <--- Alterado aqui para alinhar
com o Workbench
        user="root",
        password="",
        port=3306,
        database="almox"
    )
```

Quando API e banco estão no Compose, mude para:

```
def conectar_banco():
    return mysql.connector.connect(
        host="db",
        user="root",
```

```
password="mysql_root",  
port=3306,  
database="almox"  
)
```

Importante: Se o seu banco tiver outro usuário ou outra senha, substitua **root** e **mysql_root** pelos dados configurados no seu banco. O **host="db"** deve permanecer assim quando o banco estiver no mesmo **docker-compose.yml**.

Subir os Containers

Acesse o terminal e digite:

- **docker compose up**

Esse comando lê o arquivo **docker-compose.yml** e inicia os serviços definidos nele.

Depois, para verificar se estão funcionando, abra **outro terminal** e execute:

- **docker compose ps**

Observação

docker compose up --build

Utilize este comando quando fizer alguma alteração que afete a construção da imagem Docker. O **--build** força o Docker a reconstruir a imagem antes de iniciar os containers.

Você deve usar quando, por exemplo:

- Alterar o Dockerfile;
- Alterar o requirements.txt;
- Instalar ou adicionar uma nova biblioteca;

- Alterar algum arquivo que é copiado para dentro da imagem durante o build;
- Alterar alguma configuração utilizada durante a construção da imagem.

docker compose start: Esse comando serve somente para ligar containers que já foram criados anteriormente.

Ele é utilizado quando os containers foram apenas parados com `docker compose stop`. Se os containers foram removidos com `docker compose down`, será necessário utilizar `docker compose up` novamente.

Situação	Exemplo
Criar e iniciar os serviços	<code>docker compose up</code>
Verificar os serviços	<code>docker compose ps</code>
Ligar containers já existentes	<code>docker compose start</code>
Reconstruir a imagem e iniciar	<code>docker compose up --build</code>
Parar e remover os containers	<code>docker compose down</code>